Domain-Disguised Coding Problems — Volume 2

More business-story problems across Uber, Zomato, Google, crypto, travel, social & e-commerce

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

Read this first — evidence honesty

Volume 1 of this guide was built from a **verified research pass**, so its problems carried real tags (`[REPORTED]`, `[COMPANY-TAGGED]`). **Volume 2 is different and you should know it:**

> **Every problem here is tagged `[ILLUSTRATIVE]`.** These are realistic, well-constructed *pattern-recognition drills* — a business scenario wrapped around a known algorithm — but they are **NOT** sourced from verified interview reports. The company names show the *kind* of domain that company operates in, not a claim that they asked this exact question.

Why build them anyway? Because the skill this trains — **hearing a business story and spotting the algorithm underneath** — is exactly what the real disguised questions test, and that skill transfers regardless of which company asks. Practice de-storying on these, and the real ones become easy.

Do not walk into an interview and say "I know Uber asks X." Say instead: "problems like this map to pattern Y" — which is always true and always safe.

The de-story method (same as Volume 1)

```
1. Ignore the nouns (drivers, dishes, blocks) — decoration.
2. Find the INPUT (array? graph? intervals? stream?) and OUTPUT
   (count? min/max? boolean? all combinations? shortest path?).
3. Find the CONSTRAINT (sorted? within budget/radius? no overlaps? weighted?).
4. Match to a known pattern. The story rarely changes the pattern.
```

The problems in this volume

#	Story	Domain	Pattern
1	Match Rider to Nearest Driver	Uber / ride-hailing	heap / k-closest
2	Surge Demand Clusters	Uber / ride-hailing	grid DFS / union-find
3	Assign Orders to Fewest Riders	Zomato / delivery	interval scheduling (greedy)
4	Restaurants Within Radius	Zomato / delivery	grid / geo bucketing (BFS)
5	Search Typeahead	Google	trie + top-K

#	Story	Domain	Pattern
6	Build-Task Dependency Order	Google	topological sort
7	Detect Arbitrage Cycle	Crypto / fintech	Bellman-Ford (negative cycle)
8	Order-Matching Engine	Crypto / exchange	two heaps (priority queues)
9	Cheapest Flight, ≤ K Layovers	Travel / OTA	Bellman-Ford / BFS-with-stops
10	Best Consecutive-Nights Price	Travel / OTA	sliding window / prefix sums
11	Merge Followed Feeds	Social / streaming	merge K sorted lists (heap)
12	People You May Know	Social	graph BFS (2 levels)
13	Warehouse Robot Path	E-commerce	BFS / shortest path on grid
14	Free-Shipping Bundle	E-commerce	subset-sum / DP

1. Match Rider to Nearest Driver

> **Evidence:** [ILLUSTRATIVE — Uber / ride-hailing domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "A rider opens the app at coordinates `(x, y)`. We have a list of currently available `drivers`, each with their own `(x, y)` position on the map. Given the rider's location and the driver list, return the **K nearest drivers** to the rider, so we can offer the ride to them first. Order among the K returned drivers doesn't matter."

De-story it

Strip the map/dispatch nouns:

```
nouns to ignore: "rider", "driver", "app", "offer the ride"
INPUT   : a point (rx, ry), a list of points [(x1,y1), (x2,y2), ...], and an
          integer k
OUTPUT  : the k points from the list that are closest to (rx, ry)
CONSTRAINT: only relative distance matters, not the exact distance value;
            k can be much smaller than the number of points
```

That is **K closest points to a target point** — the classic "K closest points to the origin" problem, just shifted so the origin is the rider's location instead of `(0,0)`. Since we only need the *k smallest* distances out of *n*, and don't care about full sorted order, that's a **bounded max-heap (top-K)** pattern, not a full sort.

Worked example

```
rider = (0, 0)
drivers = [(2, 3), (1, 1), (-1, 4), (0, 5), (3, 0)]
k = 2

squared distances from rider:
  (2,3)  -> 4 + 9  = 13
  (1,1)  -> 1 + 1  = 2
  (-1,4) -> 1 + 16 = 17
  (0,5)  -> 0 + 25 = 25
  (3,0)  -> 9 + 0  = 9

two smallest: (1,1)=2, (3,0)=9
Answer: [(1,1), (3,0)]  (order doesn't matter)
```

Intuition (diagram)

```
 y
 5| (0,5)
 4| (-1,4)
 3|      (2,3)
 2|
 1|   (1,1)*
 0+---------(3,0)*----> x
   0  1  2  3
 * = the 2 nearest to rider at (0,0)
```

Keep a **max-heap of size k** ordered by distance. Push each point; once the heap holds more than `k` items, pop the farthest. Whatever survives at the end is the answer — the heap never grows past `k`, so we never pay for sorting all `n` points.

Brute force

Compute every distance, sort all `n` points by distance, take the first `k`.

```
public int[][] nearestBrute(int[] rider, int[][] drivers, int k) {
    int rx = rider[0], ry = rider[1];
    Arrays.sort(drivers, (a, b) -> {
        long da = sqDist(a, rx, ry);
        long db = sqDist(b, rx, ry);
        return Long.compare(da, db);
    });
    return Arrays.copyOfRange(drivers, 0, k);
}

private long sqDist(int[] p, int rx, int ry) {
    long dx = p[0] - rx;
    long dy = p[1] - ry;
    return dx * dx + dy * dy;   // squared distance, no sqrt needed
}
```

Optimized (bounded max-heap)

Never sort everything — maintain a max-heap capped at size `k`, keyed on squared distance (skip `sqrt`; it's monotonic so it never changes the ordering).

```
public int[][] nearestDrivers(int[] rider, int[][] drivers, int k) {
    int rx = rider[0], ry = rider[1];

    // max-heap by squared distance: farthest driver sits at the top
    PriorityQueue<int[]> heap = new PriorityQueue<>(
        (a, b) -> Long.compare(sqDist(b, rx, ry), sqDist(a, rx, ry))
    );

    for (int[] driver : drivers) {
        heap.offer(driver);
        if (heap.size() > k) {
            heap.poll();          // evict the current farthest
        }
    }

    int[][] result = new int[heap.size()][];
    for (int i = 0; i < result.length; i++) {
        result[i] = heap.poll();
    }
    return result;
}

private long sqDist(int[] p, int rx, int ry) {
    long dx = p[0] - rx;
    long dy = p[1] - ry;
    return dx * dx + dy * dy;   // use long: avoids int overflow on large coords
}
```

Version	Time	Space
Brute force (sort all)	O(n log n)	O(n)
Bounded max-heap	O(n log k)	O(k)

If the interviewer pushes further: for repeated queries against the same fixed driver set, mention a **k-d tree** or spatial grid index to answer each nearest-K query in roughly O(log n + k) instead of rescanning all drivers every time.

What to say

> "Ignoring the rider/driver framing, this is K closest points to a target — I'll shift the target to the origin and compare squared distances so I don't need `sqrt`. Since I only need the k smallest out of n, I don't sort everything; I keep a max-heap capped at size k, and any time it grows past k I pop the farthest. That's O(n log k) instead of O(n log n), and O(k) extra space. For a static, repeatedly-queried driver set, I'd reach for a k-d tree or spatial grid instead."

---

2. Surge Demand Clusters

> **Evidence:** [ILLUSTRATIVE — Uber / ride-hailing domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "We model a city as a grid of cells. You're given a matrix where each cell is either `1` (currently in **surge** — high rider demand) or `0` (normal demand). Two surge cells belong to the same **surge zone** if they're adjacent **up/down/left/right** (not diagonally). Count how many **distinct surge zones** are on the map right now. As a follow-up, also report the **size of the largest zone**, since that's the one dispatch needs to prioritize for extra drivers."

De-story it

Strip the rideshare nouns:

```
nouns to ignore: "city", "rider demand", "surge", "dispatch", "drivers"
INPUT   : an m x n grid of 0s and 1s
OUTPUT  : the count of connected components of 1s (4-directional
          adjacency), and optionally the size of the largest one
CONSTRAINT: adjacency is up/down/left/right only, not diagonal
```

That is **Number of Islands**: count connected components in a binary grid. Each `1` is land, each `0` is water, and a "surge zone" is an island. The classic pattern is **grid DFS/BFS flood fill** — walk from an unvisited `1`, mark every reachable `1` as visited, and that whole blob counts as one component. (A **union-find / disjoint-set** over grid cells is an equally valid alternative, especially if the grid is updated incrementally.)

Worked example

```
grid:
  1 1 0 0 1
  1 0 0 1 1
  0 0 0 1 0
  0 1 0 0 0

flood fill from each unvisited 1:
  zone A: (0,0),(0,1),(1,0)          -> size 3
  zone B: (0,4),(1,3),(1,4),(2,3)    -> size 4
  zone C: (3,1)                      -> size 1

Answer: 3 surge zones, largest size = 4
```

Intuition (diagram)

Land on an unvisited `1`, then spread outward to every 4-directionally connected `1`, marking each as visited so it's never counted again; each fresh unvisited `1` you land on starts a brand-new zone.

```
 1 1 0 0 1      A A . . B
 1 0 0 1 1  ->  A . . B B     3 zones: A(3) B(4) C(1)
 0 0 0 1 0      . . . B .
 0 1 0 0 0      . C . . .
```

Approach

Scan every cell; on an unvisited `1`, run DFS to mark the whole connected blob visited and tally its size, then increment the zone count.

```
public int countSurgeZones(int[][] grid) {
    if (grid == null || grid.length == 0) return 0;
    int rows = grid.length, cols = grid[0].length;
    boolean[][] visited = new boolean[rows][cols];
    int zones = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 1 && !visited[r][c]) {
                zones++;
                dfs(grid, visited, r, c);   // flood-fill this whole zone
            }
        }
    }
    return zones;
}

private void dfs(int[][] grid, boolean[][] visited, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (visited[r][c] || grid[r][c] == 0) return;
    visited[r][c] = true;
    dfs(grid, visited, r + 1, c);
    dfs(grid, visited, r - 1, c);
    dfs(grid, visited, r, c + 1);
    dfs(grid, visited, r, c - 1);
}
```

Optimized (DFS with size tracking, iterative option for large grids)

Same flood fill, but track each zone's size as we go (for the "largest zone" follow-up), and swap recursion for an explicit stack when the grid is large enough that recursion could overflow. BFS with a queue is a direct drop-in replacement with identical complexity.

```
public int[] surgeZoneStats(int[][] grid) {
    if (grid == null || grid.length == 0) return new int[]{0, 0};
    int rows = grid.length, cols = grid[0].length;
    boolean[][] visited = new boolean[rows][cols];
    int zones = 0, largest = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 1 && !visited[r][c]) {
                zones++;
                largest = Math.max(largest, floodFillIterative(grid, visited, r, c));
            }
        }
    }
    return new int[]{zones, largest};   // {zoneCount, largestZoneSize}
}

private int floodFillIterative(int[][] grid, boolean[][] visited, int startR, int startC) {
    int rows = grid.length, cols = grid[0].length;
    Deque<int[]> stack = new ArrayDeque<>();   // swap for a Queue<int[]> to get BFS
    stack.push(new int[]{startR, startC});
    visited[startR][startC] = true;
    int size = 0;
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    while (!stack.isEmpty()) {
        int[] cell = stack.pop();
        size++;
        for (int[] d : dirs) {
            int nr = cell[0] + d[0], nc = cell[1] + d[1];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            if (visited[nr][nc] || grid[nr][nc] == 0) continue;
            visited[nr][nc] = true;
            stack.push(new int[]{nr, nc});
        }
    }
    return size;
}
```

Version	Time	Space
Recursive DFS (zone count only)	O(m·n)	O(m·n) worst-case recursion stack
Iterative DFS/BFS (zone count + largest size)	O(m·n)	O(m·n) visited + O(m·n) stack/queue
Union-find alternative	O(m·n · α(m·n))	O(m·n) for parent/rank arrays

If the interviewer asks about streaming updates: when surge cells flip on/off over time rather than being given as one static snapshot, **union-find** is the better fit — union newly surging cells with their surging neighbors incrementally instead of re-scanning the whole grid. Mention it as the scale-up answer.

What to say

> "Ignoring the ride-hailing framing, this is Number of Islands: count 4-directionally connected components of 1s in a binary grid. I'll scan every cell, and on each unvisited 1 run a flood fill — DFS or BFS, same complexity either way — to mark the whole component visited and

tally its size, incrementing the zone count each time I find a fresh unvisited 1. For very large grids I'd make the flood fill iterative to avoid recursion overflow, and if the surge cells update incrementally rather than arriving as one snapshot, I'd switch to union-find so I only touch the cells that changed."

---

3. Assign Orders to the Fewest Riders

> **Evidence:** [ILLUSTRATIVE — Zomato / food-delivery domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "We run a food-delivery platform. Each order has a `pickupTime` (when the rider grabs the food from the restaurant) and a `dropoffTime` (when the rider hands it to the customer) — together an interval `[pickupTime, dropoffTime]` a rider is fully occupied with that order. A rider can only carry **one order at a time**. Given all of today's orders, what is the **minimum number of riders** we need so that every order gets picked up and dropped off without any rider double-booking?"

De-story it

Strip the delivery nouns:

```
nouns to ignore: "rider", "pickup", "dropoff", "restaurant", "customer"
INPUT   : a list of intervals [start, end], one per order
OUTPUT  : the minimum number of "workers" such that no worker is
          assigned two intervals that overlap
CONSTRAINT: a worker handles at most one interval at a time
```

That is exactly **"Meeting Rooms II" (LeetCode 253)**: the answer is the **maximum number of intervals that overlap at any single point in time**. Each rider is a "room"; we need as many rooms as the peak concurrency.

Worked example

```
orders (pickup, dropoff): [[1,4], [2,5], [7,9], [3,6]]

sweep the timeline:
  t=1: order A starts            -> concurrent = 1
  t=2: order B starts            -> concurrent = 2
  t=3: order D starts            -> concurrent = 3   <-- peak
  t=4: order A ends              -> concurrent = 2
  t=5: order B ends              -> concurrent = 1
  t=6: order D ends              -> concurrent = 0
  t=7: order C starts            -> concurrent = 1
  t=9: order C ends              -> concurrent = 0
Answer: 3 riders (needed during the [3,4) window when A, B, D all overlap)
```

Intuition (diagram)

Draw each order as a bar on a shared timeline; count how many bars are "open" at once.

```
 A: 1---------4
 B:   2----------5
 D:     3------6
 C:                 7-----9
    1  2  3  4  5  6  7  8  9
    concurrency: 1  2  3  2  1  0  1  0  0
```

Brute force

For every order, count how many other orders overlap it in time; take the max.

```
public int minRidersBrute(int[][] orders) {
    int n = orders.length;
    int maxOverlap = 0;
    for (int i = 0; i < n; i++) {
        int overlap = 0;
        for (int j = 0; j < n; j++) {
            if (orders[j][0] < orders[i][1] && orders[i][0] < orders[j][1]) {
                overlap++;             // order j is active during order i's window
            }
        }
        maxOverlap = Math.max(maxOverlap, overlap);
    }
    return maxOverlap;
}
```

Optimized (sort + two-pointer sweep)

Split pickups and dropoffs into two sorted arrays, then sweep both pointers together: each pickup before the next dropoff bumps the rider count; each dropoff frees one up.

```
public int minRiders(int[][] orders) {
    int n = orders.length;
    int[] starts = new int[n];
    int[] ends = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = orders[i][0];
        ends[i] = orders[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int riders = 0, maxRiders = 0;
    int s = 0, e = 0;
    while (s < n) {
        if (starts[s] < ends[e]) {    // a new order starts before the earliest one ends
            riders++;
            maxRiders = Math.max(maxRiders, riders);
            s++;
        } else {                      // an order finished; free that rider
            riders--;
            e++;
        }
    }
    return maxRiders;
}
```

Equivalent alternative: push dropoff times onto a **min-heap**; for each order (sorted by pickup time), if the heap's smallest dropoff is `<=` the current pickup, pop it (reuse that rider); otherwise push a new dropoff (needs a new rider). The heap size at any point is the riders in use, and its peak is the answer.

Version	Time	Space
Brute force (pairwise overlap check)	O(n^2)	O(1)
Sort + two-pointer sweep	O(n log n)	O(n)
Sort + min-heap of dropoffs	O(n log n)	O(n)

What to say

> "Ignoring the delivery framing, this is Meeting Rooms II: the minimum riders needed equals the peak number of intervals overlapping at any instant. I'll sort pickup times and dropoff times separately, then sweep with two pointers — every pickup before the next dropoff needs a fresh rider, every dropoff frees one — tracking the max concurrent count. A min-heap of active dropoff times gives the same O(n log n) bound if I need to actually assign rider IDs."

---

4. Restaurants Within Radius

> **Evidence:** [ILLUSTRATIVE — Zomato / food-delivery domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "We have a huge list of restaurants, each with a `(x, y)` coordinate. A hungry user opens the app at location `(ux, uy)` and sets a delivery radius `r`. Return every restaurant that's **within** `r` of the user. That's the easy part — now the follow-up: we have **millions of restaurants** and **thousands of these radius queries per second**. Checking every restaurant against every query is too slow. How do you make each query fast?"

De-story it

Strip the food nouns:

```
nouns to ignore: "restaurant", "user", "delivery radius"
INPUT   : a set of 2D points (the restaurant coordinates), a query
          point, and a distance r
OUTPUT  : all points whose Euclidean distance to the query point is <= r
CONSTRAINT: must scale to millions of points and many repeated queries,
          so per-query work should not touch every point
```

That is a **range / proximity search** problem: given a fixed point set, answer "which points fall in this circle?" repeatedly. Since the point set is queried over and over, it pays to **preprocess it once** into a structure that lets each query skip most of the points — this is the classic setup for **spatial bucketing / grid indexing** (with quadtrees and k-d trees as the more general cousins).

Worked example

```
restaurants (x, y): A(1,1) B(2,2) C(8,8) D(1,9) E(2,1) F(9,9)
query: user at (2,2), r = 2

brute check every point's distance to (2,2):
  A: dist 1.41  <= 2  -> keep
  B: dist 0.00  <= 2  -> keep
  C: dist 8.49  >  2  -> skip
  D: dist 7.07  >  2  -> skip
  E: dist 1.00  <= 2  -> keep
  F: dist 9.90  >  2  -> skip
Answer: [A, B, E]
```

Intuition (diagram)

Carve the plane into cells of side `r`; only check the query's cell + 8 neighbors.

```
 y  9 | D .  .  .  .  .  .  . F
    8 | .  .  .  .  .  .  .  C
      | ----+----+----+----+
    2 | .  B  . |    |    |
    1 | A  E  . |    |    |   <- query cell + 8 neighbors scanned
    0 | ----+----+----+----+
        0  1  2  3 ...     x
 cellKey = (x / r, y / r); only cells adjacent to query's cell can hold a hit
```

Brute force

Scan every restaurant, compute its distance, keep it if `<= r`.

```
public List<Restaurant> nearbyBrute(List<Restaurant> restaurants,
                                    double ux, double uy, double r) {
    List<Restaurant> result = new ArrayList<>();
    for (Restaurant rest : restaurants) {              // check ALL restaurants
        double dx = rest.x - ux;
        double dy = rest.y - uy;
        if (dx * dx + dy * dy <= r * r) {              // avoid sqrt, compare squares
            result.add(rest);
        }
    }
    return result;
}
```

Optimized (grid bucketing)

Preprocess once: drop every restaurant into a grid cell of side `r`, keyed by `(floor(x/r), floor(y/r))`. A query only needs its own cell and the 8 neighbors — any restaurant within `r` must land in one of those 9 cells.

```
class SpatialIndex {
    private final double cellSize;
    private final Map<Long, List<Restaurant>> grid = new HashMap<>();

    SpatialIndex(List<Restaurant> restaurants, double cellSize) {
        this.cellSize = cellSize;
        for (Restaurant rest : restaurants) {
            grid.computeIfAbsent(cellKey(rest.x, rest.y), k -> new ArrayList<>())
                .add(rest);
        }
    }

    private long cellKey(double x, double y) {
        long cx = (long) Math.floor(x / cellSize);
        long cy = (long) Math.floor(y / cellSize);
        return (cx << 32) ^ (cy & 0xFFFFFFFFL);        // pack (cx, cy) into one long
    }

    public List<Restaurant> nearby(double ux, double uy, double r) {
        List<Restaurant> result = new ArrayList<>();
        long cx = (long) Math.floor(ux / cellSize);
        long cy = (long) Math.floor(uy / cellSize);
        for (long dx = -1; dx <= 1; dx++) {            // own cell + 8 neighbors
            for (long dy = -1; dy <= 1; dy++) {
                long key = ((cx + dx) << 32) ^ ((cy + dy) & 0xFFFFFFFFL);
                for (Restaurant rest : grid.getOrDefault(key, Collections.emptyList())) {
                    double ddx = rest.x - ux, ddy = rest.y - uy;
                    if (ddx * ddx + ddy * ddy <= r * r) {
                        result.add(rest);
                    }
                }
            }
        }
        return result;
    }
}
```

Version	Time (per query)	Space
Brute force (scan all)	O(n)	O(1) extra
Grid bucketing	O(1 + k) average, where k = hits in the 9 cells	O(n) index, built once in O(n)

If the interviewer pushes further: grid cells work well when points are roughly uniform and `r` is fixed; if density is skewed or `r` varies a lot, mention a **quadtree** (recursively splits dense regions) or a **k-d tree** (balances on alternating dimensions) — both give O(log n + k) query time and adapt to uneven data, at the cost of a pricier build/rebalance step.

What to say

> "Ignoring the food framing, this is range search over a 2D point set with repeated queries, so it's worth preprocessing once. I'll bucket restaurants into a grid of cells sized to the query radius, keyed by `(x/r, y/r)`; each query then only scans its own cell plus the 8 neighbors instead of the whole dataset. If the radius varies widely or the data is clustered, I'd swap the fixed grid for a quadtree or k-d tree to keep queries close to O(log n)."

5. Search Typeahead

> **Evidence:** [ILLUSTRATIVE — Google / search domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "We're building the typeahead dropdown for a search box. As the user types each character, we have a `prefix` string typed so far, and a corpus of historical search terms, each with a `frequency` (how often it's been searched). Return **up to `K` matching search terms** that start with `prefix`, **ranked by popularity** (most frequent first). This needs to feel instant on every keystroke."

De-story it

Strip the search-box nouns:

```
nouns to ignore: "search box", "dropdown", "typed so far"
INPUT   : a set of strings (terms), each with an integer weight (frequency),
          plus a query string (prefix)
OUTPUT  : up to K strings that start with prefix, ordered by weight descending
CONSTRAINT: must run fast on every keystroke (repeated, incremental queries)
```

That is a **prefix-match + top-K selection** problem: find all strings sharing a prefix, then pick the K largest by weight. Repeated prefix queries on a shared string set is the classic signal for a **trie** (prefix tree), optionally with **each node caching its own top-K**.

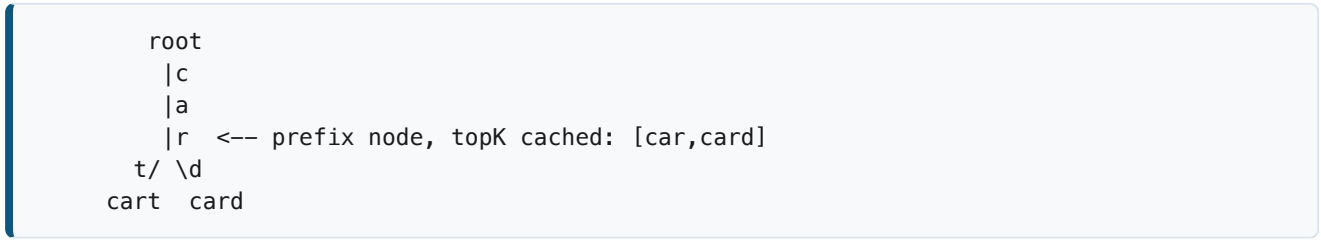
Worked example

```
terms (frequency): {"cat":50, "car":80, "cart":20, "card":65, "dog":40}
prefix = "car", K = 2

terms starting with "car": car(80), cart(20), card(65)
sorted by frequency desc:  car(80), card(65), cart(20)
top 2                    : ["car", "card"]
```

Intuition (diagram)

Walk the trie to the prefix node, then read off its top-K completions.



Brute force

Scan every term, keep the ones matching the prefix, sort all matches by frequency.

```
public List<String> topKBrute(Map<String, Integer> freq, String prefix, int k) {
    List<String> matches = new ArrayList<>();
    for (String term : freq.keySet()) {
        if (term.startsWith(prefix)) matches.add(term);
    }
    matches.sort((a, b) -> freq.get(b) - freq.get(a));   // descending by frequency
    return matches.subList(0, Math.min(k, matches.size()));
}
```

Optimized (trie with cached top-K per node)

Build a trie once. Each node caches the top-K terms in its subtree, kept sorted on insert. A query then only needs to walk `prefix.length()` characters and return the cache.

```
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    // top-K completions reachable from this node, kept sorted by frequency desc
    List<String> topK = new ArrayList<>();
}

class Typeahead {
    private final TrieNode root = new TrieNode();
    private final int k;
    private final Map<String, Integer> freq = new HashMap<>();

    Typeahead(int k) { this.k = k; }

    public void insert(String term, int frequency) {
        freq.put(term, frequency);
        TrieNode node = root;
        updateTopK(node, term);                 // root also tracks global top-K
        for (char c : term.toCharArray()) {
            node = node.children.computeIfAbsent(c, ch -> new TrieNode());
            updateTopK(node, term);              // every node on the path caches this term
        }
    }

    private void updateTopK(TrieNode node, String term) {
        node.topK.remove(term);                  // avoid duplicates on re-insert
        node.topK.add(term);
        node.topK.sort((a, b) -> freq.get(b) - freq.get(a));
        if (node.topK.size() > k) node.topK.remove(node.topK.size() - 1);
    }

    public List<String> search(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            node = node.children.get(c);
            if (node == null) return Collections.emptyList();   // no match
        }
        return node.topK;                        // O(1) lookup, precomputed
    }
}
```

Version	Time (per keystroke query)	Space
Brute force (scan + sort)	O(N + M log M), N = corpus size, M = matches	O(M) per query
Trie, cached top-K per node	O(prefixLen) query; O(L · K log K) per insert (L = term length)	O(N · L · K) worst case

If the interviewer pushes on memory: skip the per-node cache and instead walk to the prefix node in O(prefixLen), then run a small heap (size K) over just that subtree — trades query time (O(prefixLen + subtree log K)) for much less memory than caching top-K everywhere.

What to say

> "Ignoring the search-box framing, this is prefix matching plus top-K by weight, repeated on every keystroke — that repetition is what pushes me toward a trie instead of re-scanning. I'll build a trie over the terms and cache each node's top-K so a query is just a `prefixLen`-length walk. If memory is tight, I'll drop the cache and use a bounded heap over the subtree at query time instead."

---

6. Build-Task Dependency Order

> **Evidence:** [ILLUSTRATIVE — Google / build systems domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "Our build system has a set of `tasks`, each identified by an integer id. Some tasks can't start until other tasks finish — you're given a list of pairs `[a, b]` meaning `b` **must build before** `a`. Given the total number of tasks and this list of dependency pairs, produce a **valid build order** that runs every task exactly once and never runs a task before its dependencies. If the dependencies are contradictory — e.g. task 1 needs task 2 and task 2 needs task 1 — report that the build is **impossible**."

De-story it

Strip the CI/build nouns:

```
nouns to ignore: "task", "build system", "run"
INPUT   : n (number of nodes 0..n-1) and a list of directed edges
          [a, b] meaning edge b -> a ("b before a")
OUTPUT  : an ordering of all n nodes such that every edge b -> a has
          b appearing before a; or a signal that no such ordering
          exists
CONSTRAINT: dependencies form a directed graph that may or may not
          have a cycle
```

That is exactly **topological sort with cycle detection** on a directed graph — LeetCode 210, "Course Schedule II." Every "must happen before" relationship is a directed edge, and "produce a valid order or report impossible" is the textbook definition of topological sort: it exists iff the graph is a **DAG** (no cycles).

Worked example

```
n = 4 tasks: 0, 1, 2, 3
pairs [a,b] ("b before a"): [1,0], [2,0], [3,1], [3,2]

edges (b -> a):  0 -> 1,  0 -> 2,  1 -> 3,  2 -> 3
indegree:  task0=0, task1=1, task2=1, task3=2

start queue: [0]                  (only task with indegree 0)
pop 0, emit 0, decrement 1 and 2 -> indegree1=0, indegree2=0, queue=[1,2]
pop 1, emit 1, decrement 3       -> indegree3=1, queue=[2]
pop 2, emit 2, decrement 3       -> indegree3=0, queue=[3]
pop 3, emit 3

Answer: [0, 1, 2, 3]  (all 4 tasks emitted -> valid order)
```

Intuition (diagram)

Peel off nodes with no remaining incoming edges, one layer at a time.

```
   0 -> 1 -> 3          indegree0: [0]
    \        ^          after peeling 0: [1,2]
     -> 2 --/            after peeling 1,2: [3]
```

Approach (Kahn's topological sort)

Brute-forcing this would mean trying all `n!` orderings and checking each against every edge — factorial time, useless past a handful of tasks. Instead, exploit the structure: a task with **zero remaining dependencies** is always safe to run next. Track each task's **indegree** (how many prerequisites it still has). Seed a queue with every indegree-0 task, repeatedly pop one, emit it, and decrement the indegree of everything it unblocks — pushing any that drop to zero. If the emitted count ever reaches `n`, you have a full valid order; if the queue drains early, the remaining tasks are stuck in a cycle and the build is impossible.

```
public int[] buildOrder(int n, int[][] pairs) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    int[] indegree = new int[n];

    for (int[] p : pairs) {
        int a = p[0], b = p[1];          // b must build before a
        adj.get(b).add(a);               // edge b -> a
        indegree[a]++;
    }

    Deque<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) queue.add(i);
    }

    int[] order = new int[n];
    int idx = 0;
    while (!queue.isEmpty()) {
        int cur = queue.poll();
        order[idx++] = cur;
        for (int next : adj.get(cur)) {
            if (--indegree[next] == 0) queue.add(next);
        }
    }

    return idx == n ? order : new int[0];   // fewer than n emitted -> cycle, impossible
}
```

Optimized (DFS post-order alternative)

Same result, different traversal: DFS each unvisited node, recursing into its dependencies first, then appending the node to the order **after** its subtree returns (post-order). A node still "in progress" (gray) that gets revisited means a **back edge** — a cycle.

```
public int[] buildOrderDfs(int n, int[][] pairs) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] p : pairs) adj.get(p[1]).add(p[0]);   // edge b -> a

    int[] state = new int[n];             // 0=unvisited, 1=in-progress, 2=done
    int[] order = new int[n];
    int[] idx = {n - 1};                  // fill from the back (post-order)
    boolean[] hasCycle = {false};

    for (int i = 0; i < n && !hasCycle[0]; i++) {
        if (state[i] == 0) dfs(i, adj, state, order, idx, hasCycle);
    }
    return hasCycle[0] ? new int[0] : order;
}

private void dfs(int node, List<List<Integer>> adj, int[] state, int[] order,
                 int[] idx, boolean[] hasCycle) {
    state[node] = 1;                      // mark in-progress
    for (int next : adj.get(node)) {
        if (state[next] == 1) { hasCycle[0] = true; return; }   // back edge -> cycle
        if (state[next] == 0) dfs(next, adj, state, order, idx, hasCycle);
    }
    state[node] = 2;                      // mark done
    order[idx[0]--] = node;               // append to order after dependencies
}
```

Version	Time	Space
Brute force (try all orderings)	O(n! · (n + e))	O(n)
Kahn's algorithm (indegree BFS)	O(n + e)	O(n + e)
DFS post-order	O(n + e)	O(n + e)

What to say

> "Stripped of the build-system framing, this is topological sort with cycle detection: tasks are nodes, 'b before a' is a directed edge, and I need an ordering consistent with every edge — which exists iff the graph is a DAG. I'll run Kahn's algorithm: track indegree per node, seed a queue with indegree-0 nodes, and peel them off, decrementing neighbors as I go. If I emit all n nodes I have a valid order; if the queue empties early, the leftover nodes are stuck in a cycle and the build is impossible. A DFS post-order traversal with a gray/black coloring works equally well and detects cycles via back edges."

---

7. Detect an Arbitrage Cycle

> **Evidence:** [ILLUSTRATIVE — Crypto / trading domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "You're given a list of exchange rates between currencies (or tokens) — each entry says `from`, `to`, and `rate`, meaning 1 unit of `from` converts to `rate` units of `to`. Starting from any currency, is there a sequence of trades that brings you back to the **same currency** with **strictly more money than you started with**? Return `true` if such an arbitrage loop exists."

De-story it

Strip the trading nouns:

```
nouns to ignore: "currency", "exchange rate", "trade"
INPUT   : a list of weighted directed edges (from, to, rate), rate > 0
OUTPUT  : true if the graph contains a cycle whose edge weights,
          combined by multiplication, exceed 1
CONSTRAINT: "sequence of trades" = a directed walk; "back to the same
          currency" = a cycle; "more money" = product of rates > 1
```

The multiplicative condition is awkward for graph algorithms, which reason about **sums**. Take logs: `product(rate_i) > 1 <=> sum(log(rate_i)) > 0 <=> sum(-log(rate_i)) < 0`. So weight each edge `-log(rate)` and the question becomes: **does this graph contain a negative-weight cycle?** That is the textbook use case for **Bellman-Ford's negative-cycle check**.

Worked example

```
rates:
  USD -> EUR : 0.9
  EUR -> GBP : 0.8
  GBP -> USD : 1.5

product around the loop USD->EUR->GBP->USD = 0.9 * 0.8 * 1.5 = 1.08 > 1
-> trading $1 all the way around yields $1.08: arbitrage exists

transformed weights (w = -log(rate)):
  USD -> EUR : -log(0.9)  ≈  0.105
  EUR -> GBP : -log(0.8)  ≈  0.223
  GBP -> USD : -log(1.5)  ≈ -0.405
  cycle sum ≈ 0.105 + 0.223 - 0.405 = -0.077 < 0  -> negative cycle -> arbitrage
```

Intuition (diagram)

```
        0.9            0.8
   USD -----> EUR  -----> GBP
    ^                      |
    |          1.5         |
    +----------------------+
   product of rates around loop > 1  =>  negative cycle in -log space
```

Brute force

Try every simple cycle, multiply its rates, check if any product exceeds 1. Exponentially many simple cycles in a dense graph.

```
// currencies indexed 0..n-1; edges as (from, to, rate)
public boolean hasArbitrageBrute(int n, int[][] edges) {
    List<double[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] e : edges) adj[e[0]].add(new double[]{e[1], e[2]});   // (to, rate)

    for (int start = 0; start < n; start++) {
        boolean[] visited = new boolean[n];
        if (dfs(adj, start, start, 1.0, visited, true)) return true;
    }
    return false;
}

private boolean dfs(List<double[]>[] adj, int cur, int target, double product,
                    boolean[] visited, boolean isStart) {
    if (!isStart && cur == target) return product > 1.0;   // closed the loop
    if (visited[cur]) return false;
    visited[cur] = true;
    for (double[] nxt : adj[cur]) {
        if (dfs(adj, (int) nxt[0], target, product * nxt[1], visited, false)) return true;
    }
    visited[cur] = false;
    return false;
}
```

Optimized (Bellman-Ford negative-cycle detection)

Transform every edge weight to `-log(rate)`, run Bellman-Ford's relaxation `V - 1` times from a virtual source (or just seed all distances to 0, which works for cycle detection), then do **one extra pass**: if any edge still relaxes, a negative cycle — and therefore an arbitrage loop — exists.

```
public boolean hasArbitrage(int n, int[][] fromToNodes, double[] rates) {
    int m = rates.length;
    double[] weight = new double[m];
    for (int i = 0; i < m; i++) weight[i] = -Math.log(rates[i]);

    double[] dist = new double[n];        // seeding all nodes at 0 finds ANY negative cycle
    Arrays.fill(dist, 0.0);

    for (int iter = 0; iter < n - 1; iter++) {
        boolean changed = false;
        for (int e = 0; e < m; e++) {
            int u = fromToNodes[e][0], v = fromToNodes[e][1];
            if (dist[u] + weight[e] < dist[v]) {
                dist[v] = dist[u] + weight[e];
                changed = true;
            }
        }
        if (!changed) return false;        // converged early, no negative cycle
    }

    for (int e = 0; e < m; e++) {           // one more pass: still relaxing => negative cycl
        int u = fromToNodes[e][0], v = fromToNodes[e][1];
        if (dist[u] + weight[e] < dist[v]) return true;
    }
    return false;
}
```

Version	Time	Space
Brute force (enumerate cycles via DFS)	O(V · V!) worst case	O(V) recursion
Bellman-Ford negative-cycle check	O(V · E)	O(V)

If the interviewer asks for the actual cycle: track a `predecessor` array during relaxation; when the extra pass finds a relaxable edge, walk `predecessor` backward from either endpoint until you revisit a node — that revisited node marks the negative cycle, which you can then reverse into trade order.

What to say

> "Ignoring the trading framing, this is negative-cycle detection: I'll weight each edge `-log(rate)` so that a profitable loop — product of rates greater than 1 — becomes a negative-sum cycle. Bellman-Ford relaxes all edges V-1 times, and if a V-th pass can still relax an edge, there's a negative cycle, meaning an arbitrage opportunity exists. That's O(V·E), versus exponential brute-force cycle enumeration."

---

8. Order-Matching Engine

> **Evidence:** [ILLUSTRATIVE — Crypto / exchange domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "You're building the core of a crypto exchange. Traders submit `buy` and `sell` orders, each with a `price` and a `quantity`. Whenever the **highest standing buy price** is greater than or equal to the **lowest standing sell price**, those two orders should **match and execute a trade** for the overlapping quantity. Process the incoming stream of orders one at a time and report every trade that executes, along with any leftover quantity that stays on the book waiting for its next match."

De-story it

Strip the trading nouns:

```
nouns to ignore: "trader", "exchange", "order book"
INPUT   : a stream of (price, quantity, side) events, side is BUY or SELL
OUTPUT  : the list of executed (price, quantity) trades, in the order
          they occur
CONSTRAINT: a trade can only execute when best BUY price >= best SELL
            price; unmatched or partially-filled quantity remains
            pending for future matches
```

That is a **two-heap "always look at the current extreme" problem**: you repeatedly need the *maximum* buy price and the *minimum* sell price, and both change as orders arrive and get consumed. Whenever you need the running max of one set and the running min of another — recomputed after every insertion/removal — that is the **max-heap / min-heap** pattern.

Worked example

```
incoming stream (side, price, qty):
  SELL 3, 10   -> book: sells=[(3,10)]                    no match (no buys yet)
  BUY  5, 4    -> best buy 5 >= best sell 3  -> TRADE (price 3, qty 4)
                  sells=[(3,6)]  buys=[]     (sell partially filled)
  BUY  2, 5    -> best buy 2 <  best sell 3  -> no match
                  buys=[(2,5)]
  BUY  4, 10   -> best buy 4 >= best sell 3  -> TRADE (price 3, qty 6)
                  sells=[]  buys=[(4,4),(2,5)]  (buy partially filled)

executed trades: [(3,4), (3,6)]
```

Intuition (diagram)

```
   BUY heap (max on top)     SELL heap (min on top)
        9                          3
       / \                        / \
      6   5                      7   8
     /                              \
    4                                9
  pop highest buy ---> compare <--- pop lowest sell
        match while topBuy.price >= topSell.price
```

Brute force

For every incoming order, linearly scan the entire opposite-side book to find the best matching price.

```
public List<int[]> matchBrute(List<int[]> book, int[] incoming) {
    // book: list of [price, qty, side] already resting; side 0=BUY, 1=SELL
    // incoming: [price, qty, side]
    List<int[]> trades = new ArrayList<>();
    int side = incoming[2];
    int remaining = incoming[1];

    while (remaining > 0) {
        int bestIdx = -1;
        for (int i = 0; i < book.size(); i++) {          // scan the whole book, O(n)
            int[] o = book.get(i);
            if (o[2] == side || o[1] == 0) continue;      // wrong side or exhausted
            boolean crosses = (side == 0)
                    ? incoming[0] >= o[0]                  // incoming buy vs resting sell
                    : incoming[0] <= o[0];                 // incoming sell vs resting buy
            if (!crosses) continue;
            if (bestIdx == -1
                    || (side == 0 && o[0] < book.get(bestIdx)[0])
                    || (side == 1 && o[0] > book.get(bestIdx)[0])) {
                bestIdx = i;                                // track best price, O(n) per can
            }
        }
        if (bestIdx == -1) break;                           // nothing crosses -> rest on book
        int[] match = book.get(bestIdx);
        int fillQty = Math.min(remaining, match[1]);
        trades.add(new int[]{match[0], fillQty});
        match[1] -= fillQty;
        remaining -= fillQty;
    }
    if (remaining > 0) book.add(new int[]{incoming[0], remaining, side});
    return trades;                                          // O(n) per order -> O(n^2) overa
}
```

Optimized (two heaps: max-heap of buys, min-heap of sells)

Keep the best buy and best sell always at the top of their heap. On each new order, push it onto the correct heap, then repeatedly match the tops while they cross, filling partially and popping only what's exhausted.

```
private final PriorityQueue<int[]> buys =                    // max-heap by price
        new PriorityQueue<>((a, b) -> Integer.compare(b[0], a[0]));
private final PriorityQueue<int[]> sells =                   // min-heap by price
        new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
// each entry: [price, quantity]

public List<int[]> match(int price, int quantity, boolean isBuy) {
    List<int[]> trades = new ArrayList<>();
    int remaining = quantity;

    PriorityQueue<int[]> own = isBuy ? buys : sells;
    PriorityQueue<int[]> opposite = isBuy ? sells : buys;

    while (remaining > 0 && !opposite.isEmpty()) {
        int[] best = opposite.peek();                        // O(1) look at best opposite pr
        boolean crosses = isBuy ? price >= best[0] : price <= best[0];
        if (!crosses) break;                                  // no more matches possible

        int fillQty = Math.min(remaining, best[1]);
        trades.add(new int[]{best[0], fillQty});              // trade executes at resting p
        remaining -= fillQty;
        best[1] -= fillQty;
        if (best[1] == 0) opposite.poll();                    // O(log n): fully filled, rem
    }

    if (remaining > 0) own.offer(new int[]{price, remaining}); // O(log n): rest leftover on
    return trades;
}
```

Version	Time	Space
Brute force (linear scan per order)	O(n^2) total for n orders	O(n)
Two heaps (max-heap buys, min-heap sells)	O(log n) per order, O(n log n) total	O(n)

If the interviewer pushes further: note that `Integer.compare` avoids the classic `a[0] - b[0]` overflow bug in comparators, and that carrying an `orderId` in each heap entry (with a lazy-deletion `Set<Integer>` of canceled ids) extends this cleanly to order cancellation without rebuilding the heap.

What to say

> "Ignoring the trading framing, I always need the current max of one set and current min of another, both changing as elements are added and removed — that's the two-heap pattern. I'll keep a max-heap of buy orders and a min-heap of sell orders, and on every new order I match against the opposite heap's top while prices cross, partially filling and popping only when a resting order is exhausted. That's O(log n) per order instead of an O(n) rescan, so O(n log n) for the whole stream."

---

9. Cheapest Flight with at most K Layovers

> **Evidence:** [ILLUSTRATIVE — Travel / OTA domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "You're given a list of flight routes — each entry is `from`, `to`, and `price` for a direct flight between two cities. Given a source city, a destination city, and a maximum number of `K` layovers (stops in between), find the **cheapest total price** to fly from source to destination using **at most K layovers**. If no such route exists, return `-1`."

De-story it

Strip the travel nouns:

```
nouns to ignore: "flight", "city", "layover", "price"
INPUT   : a weighted directed graph (from, to, price) plus a source
          node, a destination node, and an integer K
OUTPUT  : the minimum-cost path from source to destination that uses
          at most K+1 edges (K layovers = K intermediate stops)
CONSTRAINT: "at most K layovers" caps the number of edges in the path,
          not just the total cost
```

That is **shortest weighted path with a hop budget**: normal shortest-path algorithms don't track "how many edges have I used so far," so you need a variant that adds that extra dimension to the state.

Worked example

```
flights (from, to, price):
  0 -> 1 : 100
  1 -> 2 : 100
  0 -> 2 : 500
  1 -> 3 : 600
  2 -> 3 : 200
src = 0, dst = 3, K = 1  (at most 1 layover, i.e. at most 2 edges)

candidate paths within the K-layover budget:
  0->1->3      : 100 + 600 = 700   (1 layover: city 1)
  0->2->3      : 500 + 200 = 700   (1 layover: city 2)
  0->1->2->3   : 100+100+200 = 400  <- cheapest, but uses 2 layovers, TOO MANY
Answer: 700  (the 3-edge path is cheaper but violates the layover cap)
```

Intuition (diagram)

```
        100        100        200
   0 -------> 1 -------> 2 -------> 3
   |                               ^
   +---------------500--------------+
   cheapest overall path (0–1–2–3) breaks the K=1 layover budget
```

Brute force

DFS every path from source to destination, but stop branching once the number of layovers used exceeds K. Exponentially many paths in a dense graph.

```
public int cheapestBrute(int n, int[][] flights, int src, int dst, int k) {
    List<int[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] f : flights) adj[f[0]].add(new int[]{f[1], f[2]});   // (to, price)

    int[] best = {Integer.MAX_VALUE};
    dfs(adj, src, dst, k, 0, best);
    return best[0] == Integer.MAX_VALUE ? -1 : best[0];
}

private void dfs(List<int[]>[] adj, int cur, int dst, int stopsLeft, int cost, int[] best) {
    if (cur == dst) { best[0] = Math.min(best[0], cost); return; }
    if (stopsLeft < 0) return;                 // ran out of layover budget
    for (int[] nxt : adj[cur]) {
        int to = nxt[0], price = nxt[1];
        if (cost + price < best[0]) {          // light pruning
            dfs(adj, to, dst, stopsLeft - 1, cost + price, best);
        }
    }
}
```

Optimized (Bellman-Ford limited to K+1 rounds)

Run Bellman-Ford's edge relaxation, but cap it at exactly `K + 1` rounds — each round represents allowing one more edge (hop) into a path. The critical trick: relax from a **snapshot** of the previous round's distances, so a city updated earlier in this round can't immediately feed a second edge in the same round (which would silently use up an extra hop for free).

```
public int cheapestFlight(int n, int[][] flights, int src, int dst, int k) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    for (int round = 0; round <= k; round++) {      // at most K+1 edges allowed
        int[] snapshot = dist.clone();               // freeze this round's starting distance
        for (int[] f : flights) {
            int from = f[0], to = f[1], price = f[2];
            if (snapshot[from] == Integer.MAX_VALUE) continue;   // unreachable so far
            if (snapshot[from] + price < dist[to]) {
                dist[to] = snapshot[from] + price;    // only one hop's worth of update this
            }
        }
    }
    return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
}
```

Version	Time	Space
Brute force (DFS all paths, prune by stops)	O(E^K) worst case	O(K) recursion
Bellman-Ford, K+1 rounds	O(K · E)	O(V)

Why not plain Dijkstra? Dijkstra's greedy "settle the closest node once" invariant assumes more edges never hurt, so it has no notion of a hop budget. It can be adapted by expanding the state to `(cost, node, stopsUsed)` and pushing every reachable `(neighbor, stopsUsed+1)` even if the neighbor was visited with fewer stops before — that recovers correctness at the cost of Dijkstra's usual efficiency, so for small K, Bellman-Ford's round-capped relaxation is simpler and equally fast.

What to say

> "Ignoring the travel framing, this is shortest path with a hop cap: at most K layovers means at most K+1 edges in the path. I'll run Bellman-Ford relaxation for exactly K+1 rounds, using a snapshot of distances at the start of each round so I don't accidentally chain two edges together within a single round. That's O(K·E), and it naturally returns -1 if the destination is still unreachable after K+1 rounds. Plain Dijkstra doesn't work directly here because it has no concept of how many edges a path has used, so it would need an extra stops dimension in its state to be adapted."

---

10. Best Consecutive-Nights Price

> **Evidence:** [ILLUSTRATIVE — Travel / OTA domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "We show a nightly price calendar for a hotel — one price per night for the next `n` nights. A guest wants to book a stay of exactly `k` consecutive nights and, naturally, wants to pay as little as possible in total. Given the price calendar and `k`, find the **check-in night that starts the cheapest possible `k`-night stay**, and report that total price."

De-story it

Strip the travel nouns:

```
nouns to ignore: "hotel", "guest", "check-in", "stay"
INPUT   : an array of positive integers (nightly prices), and an
          integer k (stay length)
OUTPUT  : the minimum sum of any k consecutive elements, and the
          start index where that minimum occurs
CONSTRAINT: the window size is fixed at k; window must stay
            contiguous within the array bounds
```

That is a **fixed-size sliding window / minimum subarray sum** problem: slide a window of length `k` across the array and track the smallest sum seen.

Worked example

```
prices = [120, 90, 150, 60, 40, 80]   k = 3

windows of size 3:
  [120,90,150] = 360   start=0
  [90,150,60]  = 300   start=1
  [150,60,40]  = 250   start=2
  [60,40,80]   = 180   start=3   <-- best
Answer: total = 180, check-in night = index 3
```

Intuition (diagram)

```
[120 90 150 60 40 80]
 |--w--|                sum=360
    |--w--|              sum=300
       |--w--|           sum=250
          |--w--|        sum=180 *min
```

Brute force

For every possible start, sum the next `k` nights from scratch.

```
public int[] cheapestStayBrute(int[] prices, int k) {
    int n = prices.length;
    int bestSum = Integer.MAX_VALUE;
    int bestStart = -1;
    for (int start = 0; start <= n - k; start++) {   // every valid start
        int sum = 0;
        for (int i = start; i < start + k; i++) {    // re-sum the whole window
            sum += prices[i];
        }
        if (sum < bestSum) {
            bestSum = sum;
            bestStart = start;
        }
    }
    return new int[] { bestStart, bestSum };
}
```

Optimized (sliding window)

Compute the sum of the first window once, then slide by one night at a time: **add the incoming night, subtract the outgoing night**. No re-summing.

```
public int[] cheapestStay(int[] prices, int k) {
    int n = prices.length;
    int windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += prices[i];               // build the first window
    }
    int bestSum = windowSum;
    int bestStart = 0;

    for (int start = 1; start <= n - k; start++) {
        windowSum += prices[start + k - 1];   // add the night entering the window
        windowSum -= prices[start - 1];       // drop the night leaving the window
        if (windowSum < bestSum) {
            bestSum = windowSum;
            bestStart = start;
        }
    }
    return new int[] { bestStart, bestSum };
}
```

Prefix-sum variant: build `prefix[i] = prices[0] + ... + prices[i-1]`, then any window sum is `prefix[start + k] - prefix[start]` in O(1) per window — same O(n) total, useful if you need many arbitrary-range sums later, not just size-k ones.

Version	Time	Space
Brute force (re-sum each window)	O(n·k)	O(1)
Sliding window	O(n)	O(1)

Version	Time	Space
Prefix sums	O(n) build, O(1) per query	O(n)

What to say

> "Dropping the hotel framing, this is minimum sum of a fixed-size subarray: I need the cheapest `k`-length window over the price array. Instead of re-summing each window from scratch, I'll slide it — add the night coming in, subtract the night falling out — which turns an O(n·k) scan into O(n). If I needed sums over many different, non-fixed ranges later, I'd reach for prefix sums instead."

---

11. Merge Followed Feeds

> **Evidence:** [ILLUSTRATIVE — Social / streaming domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "A user follows several other accounts. Each followed account's posts arrive to you as a **list already sorted by timestamp**. Given the `K` lists of that user's followed accounts, produce **one merged timeline** — all posts across every followed account, in overall sorted order by timestamp."

De-story it

Strip the social nouns:

```
nouns to ignore: "followed account", "post", "timeline"
INPUT   : K lists of integers (timestamps), each list individually sorted
OUTPUT  : one list containing every integer from every input list,
          in fully sorted order
CONSTRAINT: each of the K lists is already sorted on its own;
            lists may differ in length
```

That is **merge K sorted lists**, i.e. LeetCode 23 ("Merge k Sorted Lists"). The "already sorted per list" clause is the tell: you never need to re-sort within a list, you only need to repeatedly pick the smallest available head across lists → **min-heap of list heads**.

Worked example

```
3 followed accounts, timestamps already sorted per account:
  feed A: [1, 4, 9]
  feed B: [2, 5]
  feed C: [3, 6, 7]

merged timeline (sorted): [1, 2, 3, 4, 5, 6, 7, 9]

heap trace (value, feed):
  push heads (1,A) (2,B) (3,C)
  pop (1,A) -> push next head of A: (4,A)   output: 1
  pop (2,B) -> push next head of B: (5,B)   output: 1,2
  pop (3,C) -> push next head of C: (6,C)   output: 1,2,3
  pop (4,A) -> A exhausted after this        output: 1,2,3,4
  pop (5,B) -> B exhausted after this        output: 1,2,3,4,5
  pop (6,C) -> push next head of C: (7,C)   output: 1,2,3,4,5,6
  pop (7,C) -> C exhausted                   output: 1,2,3,4,5,6,7
  pop (9,A) -> A exhausted                   output: 1,2,3,4,5,6,7,9
```

Intuition (diagram)

```
 A: 1->4->9      \
 B: 2->5          }--> min-heap of K heads --> merged
 C: 3->6->7->     /      (pop min, push its next)
```

Brute force

Dump every post from every feed into one list, then sort it.

```
public List<Integer> mergeBrute(List<List<Integer>> feeds) {
    List<Integer> all = new ArrayList<>();
    for (List<Integer> feed : feeds) {
        all.addAll(feed);          // flatten every followed account's posts
    }
    Collections.sort(all);         // O(N log N) over all N posts
    return all;
}
```

Optimized (min-heap over K feed heads)

Never look at more than one "current" post per feed at a time. Keep a min-heap of the current head of each of the K feeds; pop the smallest, output it, and push that feed's next post (if any).

```
public List<Integer> mergeOptimized(List<List<Integer>> feeds) {
    // heap entry: [timestamp, feedIndex, postIndexWithinFeed]
    PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> a[0] - b[0]);

    for (int f = 0; f < feeds.size(); f++) {
        if (!feeds.get(f).isEmpty()) {
            heap.offer(new int[]{feeds.get(f).get(0), f, 0});   // seed with each feed's head
        }
    }

    List<Integer> merged = new ArrayList<>();
    while (!heap.isEmpty()) {
        int[] cur = heap.poll();                 // smallest timestamp across all feeds
        int ts = cur[0], f = cur[1], i = cur[2];
        merged.add(ts);

        List<Integer> feed = feeds.get(f);
        if (i + 1 < feed.size()) {                // this feed has a successor post
            heap.offer(new int[]{feed.get(i + 1), f, i + 1});
        }
    }
    return merged;
}
```

Version	Time	Space
Brute force (flatten + sort)	O(N log N)	O(N)
Min-heap over K heads	O(N log K)	O(K) heap + O(N) output

If the interviewer asks about linked-list feeds instead of arrays: same heap trick, just store `PostNode` (with a `next` pointer) in the heap instead of `(feed, index)`, and push `node.next` after popping — the complexity is identical.

What to say

> "Ignoring the social framing, this is merge-K-sorted-lists: each followed account's posts arrive pre-sorted, so I never need to sort within a feed, only pick the smallest available head across feeds. I'll keep a min-heap seeded with each feed's current head, pop the smallest, emit it, and push that feed's next post — O(N log K) instead of the O(N log N) brute-force sort of everything."

---

12. People You May Know

> **Evidence:** [ILLUSTRATIVE — Social network domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "Our social app models users and their friendships as a network — each user has a list of direct friends. For a given user, build a **'People You May Know'** feed: candidates are people who are **friends of the user's friends**, but are **not** already a direct friend (and not the user themself). Rank the candidates by **how many mutual friends** they share with the user, highest first."

De-story it

Strip the social nouns:

```
nouns to ignore: "user", "friend", "feed", "social network"
INPUT   : an undirected graph of nodes (people) and edges (friendships),
          plus a source node
OUTPUT  : all nodes at exactly graph-distance 2 from the source (excluding
          the source and its distance-1 neighbors), ranked by the number
          of distance-1 neighbors they share with the source
CONSTRAINT: exclude the source node itself and any existing direct
            neighbors; "mutual friends" = count of shared distance-1 nodes
```

That is a **2-hop breadth-first search with a frequency count**: expand one layer out from the source's neighbors, tally how many times each newly-seen node is reached, and that tally *is* the mutual-friend count — no separate intersection step needed once you frame it as BFS.

Worked example

```
adjacency (friendships):
  A - B, A - C, A - D
  B - E, B - F
  C - E
  D - F, D - G

Query: people you may know for A
A's direct friends: {B, C, D}   (excluded from results)

2-hop expansion from B, C, D:
  via B -> E, F
  via C -> E
  via D -> F, G

mutual-friend tally (candidate -> count):
  E : reached via B, C   -> 2
  F : reached via B, D   -> 2
  G : reached via D      -> 1

Answer (ranked): [E (2), F (2), G (1)]
```

Intuition (diagram)

```
    A --- B --- E     A's friends: B,C,D
    |     |           2-hop via them: E,F,G
    C-----+--- E      E: via B,C (2)  <- top
    |     |           F: via B,D (2)  <- top
    D-----+--- F      G: via D   (1)
    |
    +---------- G
```

Brute force

For every other node in the graph, intersect its friend set with the source's friend set and count the overlap.

```
public List<Integer> suggestBrute(Map<Integer, Set<Integer>> graph, int user) {
    Set<Integer> myFriends = graph.get(user);
    Map<Integer, Integer> mutualCount = new HashMap<>();

    for (int candidate : graph.keySet()) {           // every user in the network
        if (candidate == user || myFriends.contains(candidate)) continue;
        Set<Integer> theirFriends = graph.get(candidate);
        int shared = 0;
        for (int f : myFriends) {                     // intersect friend sets
            if (theirFriends.contains(f)) shared++;
        }
        if (shared > 0) mutualCount.put(candidate, shared);
    }

    List<Integer> result = new ArrayList<>(mutualCount.keySet());
    result.sort((a, b) -> mutualCount.get(b) - mutualCount.get(a));
    return result;
}
```

Optimized (2-hop BFS with a tally map)

Only walk out from the user's own friends, two hops — never touch nodes elsewhere in the network.

```
public List<Integer> suggest(Map<Integer, Set<Integer>> graph, int user) {
    Set<Integer> myFriends = graph.get(user);
    Map<Integer, Integer> mutualCount = new HashMap<>();

    for (int friend : myFriends) {                    // hop 1: user's direct friends
        for (int candidate : graph.get(friend)) {      // hop 2: their friends
            if (candidate == user || myFriends.contains(candidate)) continue;
            mutualCount.merge(candidate, 1, Integer::sum);  // tally = mutual-friend count
        }
    }

    List<Integer> result = new ArrayList<>(mutualCount.keySet());
    result.sort((a, b) -> mutualCount.get(b) - mutualCount.get(a));  // rank by mutual count
    return result;
}
```

Version	Time	Space
Brute force (scan all users, intersect sets)	O(V · F)	O(V)
2-hop BFS with tally map	O(F^2) where F = avg friends per user	O(F) candidates

If the interviewer pushes for top-K instead of a full ranking: drop the sort and use a min-heap of size K while tallying, giving O(F^2 · log K) instead of O(F^2 log F) for the sort.

What to say

> "Ignoring the social framing, this is a 2-hop BFS from the source node: expand each direct neighbor's neighbors, tally how many times each new node is reached in a hashmap, and that tally is exactly the mutual-friend count. I only need to touch the user's own friends' friends, not the whole graph, so it's O(F^2) instead of scanning every user. If they want just the top K suggestions, I'd swap the final sort for a small min-heap."

13. Warehouse Robot Path

> **Evidence:** [ILLUSTRATIVE — E-commerce / warehouse domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "Our fulfillment center floor is modeled as a grid. A picking robot starts at position `S` and needs to reach a packing station at `T`. Some cells are storage shelves — the robot **cannot drive through them**. The robot moves one cell at a time, up/down/left/right, and each move takes one time unit. Return the **minimum number of moves** for the robot to reach `T`, or `-1` if it's boxed in and can never get there."

De-story it

Strip the warehouse nouns:

```
nouns to ignore: "robot", "shelves", "packing station", "fulfillment center"
INPUT   : an m x n grid of cells, each either open or blocked; a start
          cell S and a target cell T
OUTPUT  : the minimum number of 4-directional unit steps from S to T
          moving only through open cells; -1 if T is unreachable
CONSTRAINT: every move costs exactly 1 (unweighted graph)
```

That is **shortest path on an unweighted graph** — the grid's open cells are nodes, edges connect 4-directional neighbors with weight 1. Unweighted shortest path is the textbook signal for **BFS**: explore level by level, and the first time you reach `T` you've found the shortest distance, because BFS visits all nodes at distance `k` before any node at distance `k+1`.

Worked example

```
grid (S=start, T=target, #=shelf, .=open floor), 5 rows x 5 cols:

S . . # .
. # . # .
. # . . .
. # # # .
. . . . T

BFS distances from S (steps to reach each open cell):
0 1 2 . 6
1 . 3 . 7
2 . 4 5 6
3 . . . 7
4 5 6 7 8

Answer: 8 steps (there is exactly one open corridor around the shelf block)
```

Intuition (diagram)

BFS grows a "wavefront" outward from S one ring at a time; shelves just block a cell from ever entering the queue.

```
ring0: S
ring1: neighbors of S
ring2: neighbors of ring1 not yet visited
ring3: ...
-> T is first touched on the ring whose number IS the answer

. . 3 . .      3 = cells reachable in exactly 3 moves
. 2 2 2 .      2 = cells reachable in exactly 2 moves
3 2 S 2 3      S = start, 1-ring and 2-ring expand outward
. 2 2 2 .
. . 3 . .
```

Brute force

DFS every path from S, exploring every possible route and tracking the shortest one seen. It revisits the same cell via many different paths, so it is exponential and easy to get wrong for "shortest" without extra bookkeeping.

```
private int best = Integer.MAX_VALUE;

public int shortestPathBrute(char[][] grid, int[] start, int[] target) {
    boolean[][] visited = new boolean[grid.length][grid[0].length];
    dfs(grid, start[0], start[1], target, visited, 0);
    return best == Integer.MAX_VALUE ? -1 : best;
}

private void dfs(char[][] grid, int r, int c, int[] target,
                 boolean[][] visited, int steps) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] == '#' || visited[r][c] || steps >= best) return;   // blocked/pruned

    if (r == target[0] && c == target[1]) { best = Math.min(best, steps); return; }

    visited[r][c] = true;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int[] d : dirs) {
        dfs(grid, r + d[0], c + d[1], target, visited, steps + 1);     // try every path
    }
    visited[r][c] = false;                                            // undo for other paths
}
```

Optimized (BFS with a visited grid)

Push `S` onto a queue with distance 0. Pop, and for each unvisited open neighbor, mark it visited and push it with `distance + 1`. Because BFS expands ring by ring, the first pop of `T` carries the shortest distance — stop immediately.

```
public int shortestPath(char[][] grid, int[] start, int[] target) {
    int rows = grid.length, cols = grid[0].length;
    boolean[][] visited = new boolean[rows][cols];
    Queue<int[]> queue = new LinkedList<>();     // {row, col, dist}
    queue.offer(new int[]{start[0], start[1], 0});
    visited[start[0]][start[1]] = true;

    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};

    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        if (cur[0] == target[0] && cur[1] == target[1]) return cur[2];   // first hit = short

        for (int[] d : dirs) {
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            if (grid[nr][nc] == '#' || visited[nr][nc]) continue;
            visited[nr][nc] = true;                                    // mark before enqueu
            queue.offer(new int[]{nr, nc, cur[2] + 1});
        }
    }
    return -1;   // queue drained without reaching target
}
```

Version	Time	Space
Brute force (DFS all paths)	exponential, O(4^(m·n)) worst case	O(m·n) recursion
BFS with visited grid	O(m · n)	O(m · n)

If the interviewer pushes for scale: on a huge grid where you only care about one start-target pair (not all distances), swap BFS for **A*** using Manhattan distance to `T` as the heuristic — it still guarantees the shortest path on this uniform-cost grid but explores far fewer cells in practice.

What to say

> "Ignoring the warehouse framing, this is shortest path on an unweighted grid graph, so BFS is the right tool: expand outward ring by ring from S, mark cells visited as I enqueue them so I never revisit, and return the distance the moment I pop T — that's guaranteed shortest because BFS processes closer cells first. That's O(m·n) time and space. If the grid were huge and I only needed one S-to-T query, I'd switch to A* with a Manhattan-distance heuristic to prune the search."

---

14. Free-Shipping Bundle

> **Evidence:** [ILLUSTRATIVE — E-commerce / cart domain. A realistic pattern-recognition drill, NOT a verified interview report.]

How the interviewer poses it. "A shopper's cart is `remaining` dollars short of free shipping. We show a list of eligible **add-on items**, each with a fixed `price`. Can we choose a **subset of add-ons** whose prices **sum to exactly** `remaining`, so the shopper unlocks free shipping without paying for anything extra? Each add-on can be added **at most once**. Return `true` / `false` — and if `true`, one such combination."

De-story it

Strip the retail nouns:

```
nouns to ignore: "cart", "add-on", "shopper", "free shipping"
INPUT   : a list of positive integers (add-on prices) and a target
          integer `remaining`
OUTPUT  : true if some subset of the prices sums to EXACTLY `remaining`
          (plus one witnessing subset); false otherwise
CONSTRAINT: each price usable at most once (0/1 choice per item)
```

That is **subset-sum as a decision problem** — the same shape as LeetCode 416 ("Partition Equal Subset Sum"): choose a subset of items to hit an exact target. Each add-on is used at most once → 0/1 choice per item → **classic 0/1 subset-sum DP**.

Worked example

```
add-on prices: [5, 3, 9, 2]
remaining = 10

candidate subsets:
  [5,3,2]   = 10   <-- exact match, answer true
  [9,2]     = 11   (overshoot)
  [5,3]     = 8    (undershoot)
  [9]       = 9    (undershoot)
Answer: true, e.g. {5, 3, 2}
```

Intuition (diagram)

```
dp[s] = can some subset of prices seen so far sum to s?
dp[0]=T, else F.  For each price p: dp[s] |= dp[s-p], s: T..p
```

Brute force

Enumerate **all** 2^n subsets, check if any sums to exactly `remaining`.

```
public boolean canUnlockBrute(int[] prices, int remaining) {
    int n = prices.length;
    for (int mask = 0; mask < (1 << n); mask++) {   // every subset
        int sum = 0;
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) sum += prices[i];
        }
        if (sum == remaining) return true;
    }
    return false;
}
```

Optimized (0/1 subset-sum DP)

`dp[s]` = true if some subset of the prices examined so far sums to exactly `s`. Process each price once, and update `dp` from **high sum to low sum** so each item is only used once per row (the standard 0/1 knapsack trick — updating low-to-high would let an item be reused).

```
public boolean canUnlock(int[] prices, int remaining) {
    boolean[] dp = new boolean[remaining + 1];
    dp[0] = true;                          // sum of 0 is always reachable (empty subset)

    for (int price : prices) {
        for (int s = remaining; s >= price; s--) {   // high -> low: use price at most once
            if (dp[s - price]) dp[s] = true;
        }
        if (dp[remaining]) return true;    // early exit once target is reachable
    }
    return dp[remaining];
}
```

Version	Time	Space
Brute force (all subsets)	O(2^n · n)	O(n)
0/1 subset-sum DP	O(n · remaining)	O(remaining)

If the interviewer varies it: "reach *at least* `remaining`, minimizing overshoot" is the same DP with the goal changed — track the smallest reachable sum `>= remaining` instead of an exact match, or extend `dp[s]` to store the minimum item count for each reachable sum.

What to say

> "Ignoring the cart framing, this is 0/1 subset-sum: can a subset of prices sum to exactly the target? I'll build a boolean `dp[s]` where `dp[0]` is true and, for each price, update `dp` from high sum down to low sum so every item is used at most once — that's the standard 0/1 knapsack ordering. That gives O(n · remaining) time instead of enumerating all 2^n subsets. If the ask shifts to 'reach at least the target with minimal overshoot,' I'd track the closest reachable sum instead of an exact hit."